```
    cy.get("[data-cy=ticketCost").should("have.text", "$30.00")
  })

  it("undoes a sale on second click", () => {
    cy.get("[data-cy=1x10]").click()
    cy.get("[data-cy=1x10]").should("have.attr", "data-color", "green")
    cy.get("[data-cy=1x10]").click()
    cy.get("[data-cy=1x10]").should("have.attr", "data-color", "white")
    cy.get("[data-cy=ticketsPurchased").should("have.text", "0")
    cy.get("[data-cy=ticketCost").should("have.text", "$0.00")
  })

  it("clears from the clear ticket button", () => {
    cy.get("[data-cy=1x10]").click()
    cy.get("[data-cy=1x4]").click()
    cy.get("[data-cy=1x3]").click()
    cy.get("[data-cy=clearButton]").click()
    cy.get("[data-cy=1x10]").should("have.attr", "data-color", "white")
    cy.get("[data-cy=ticketsPurchased").should("have.text", "0")
    cy.get("[data-cy=ticketCost").should("have.text", "$0.00")
  })
})
```

Our beforeEach function starts each test by resetting, then visiting a specific concert page. I'm using a little hack here to get around a different problem, which is that the JavaScript code doesn't know the ID of any of the concerts that the database seed creates. There are a few different ways around this—you could explicitly set the ID in the seed file, or you could make the URLs work with a friendly slug that would be the same each time. I did neither of these things, instead special casing the route concerts/last to grab the last concert in the system:

chapter_15/02/app/controllers/concerts_controller.rb

```
def set_concert
  params[:id] = Concert.last.id if params["id"] == "last"
  @concert = Concert.includes(:tickets).find(params[:id])
end
```

The first test checks that seats become invalid after the number of tickets to buy at a time changes. To make this work, I added a data-cy attribute to the "tickets to buy" pull-down menu and a few other data-cy attributes in the subtotal and header,